

Mohit Arora

The Art of Hardware Architecture

Design Methods and Techniques
for Digital Circuits



Springer

Mohit Arora

The Art of Hardware Architecture

Design Methods and Techniques
for Digital Circuits

 Springer

Mohit Arora
Freescale Semiconductor
Faridabad
India
mohit.arora@mc.com

ISBN 978-1-4614-0396-8 e-ISBN 978-1-4614-0397-5
DOI 10.1007/978-1-4614-0397-5
Springer New York Dordrecht Heidelberg London

Library of Congress Control Number: 2011934353

© Springer Science+Business Media, LLC 2012

All rights reserved. This work may not be translated or copied in whole or in part without the written permission of the publisher (Springer Science+Business Media, LLC, 233 Spring Street, New York, NY 10013, USA), except for brief excerpts in connection with reviews or scholarly analysis. Use in connection with any form of information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed is forbidden.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

*This book is dedicated to my wife Pooja
and my daughter Prisha.*

Preface

I started my Industry career in year 2000 in the field of Chip Design. My work involved lot of research that gave me opportunity to write technical papers, participate in various conferences and share practical experiences. During this journey I got lot of positive feedback on my publications. Readers have often asked me forcing me to think if I should write a book compiling all the practical experiences. The book's aim is to highlight all the complex issues, tasks and skills that must be mastered by an IP designer to design an optimized and robust digital circuit to solve a problem. The techniques and methodologies prescribed in the book, if properly employed, can significantly reduce the time it takes to convert initial ideas and concepts into right-first-time silicon.

The book is intended for a wide audience. Though it may be used in an undergraduate or graduate course, book is mainly intended for those in semiconductor industries who are directly involved with chip design and requires deeper understand of the subject.

This book is distinguished from others by its primary focus on real problems rather than theoretical concepts with its emphasis on design techniques across various aspects of chip-design.

The book covers aspects of chip design in a consistent way, starting with basic concepts in Chap. 1 and gradually increasing the depth to reach advanced concepts, such as EMC design techniques or sophisticated low power techniques like DVFS (Dynamic Voltage and Frequency scaling).

Chapter 1 covers “*metastability*” to help user understand more clearly the issues related to metastability, how it can be quantified and necessary techniques to minimize its effort.

Chapter 2 covers general set of recommendations around “*clocks and resets*” intended for use by designers while designing a block or Intellectual Property (IP). The guidelines are independent of any CAD tool or silicon process and are applicable to any ASIC designs.

Chapter 3 goes beyond synchronous clock designs and covers asynchronous clocks or “*handling multiple clocks*” in design, problems faced and solutions in order to get a robust designs that works on multiple clocks.

Chapter 4 covers all about “*Clock Dividers*” that a typical SoC may require generating number of phase related clocks. Apart from synchronous division where required clocks are generated by dividing the master clock by a power of two, chapter also covers odd division (Divide by 3, 5 etc.) as well as non-integer dividers (Divide by 1.5, 2.5 etc.).

Chapter 5 covers all about “*Low Power Design techniques*”. In recent times, power consumption has become a significant design constraint with shrinking technology as well as to meet power targets for energy efficient applications. This Chapter describes various design methodologies and techniques at various levels of design abstraction to reduce dynamic and as well as static power consumption.

Chapter 6 covers the concept of “*Pipelining*”, the way it applies to processor design to increase the throughput in terms of calculations per clock cycle. The chapter extends the scope of pipelining beyond microprocessor to cover typical circuits so as to increase performance.

Chapter 7 covers “*Endianess issues*” in design that may include several third-party IPs with different Endianess and the way it can be handled in the design in an optimal way.

Chapter 8 covers several hardware as well as software “*Debouncing Techniques*” to eliminate unwanted noise or glitch in the circuit caused by an external input (usually some kind of switch).

Chapter 9 covers deep details on EMC/EMI issues, the way it applies to digital circuits and design guidelines that can be followed at various level of abstraction for “*better EMC performance*”.

Theoretical part has been intentionally kept to the minimum that is essentially required to understand the subject. The guidelines explained across various chapters are independent of any CAD tool or silicon process and are applicable to any ASIC designs and can help designers to plan and to execute a successful System on Chip (SoC) with a well-structured and synthesizable RTL code.

There are few chapters that include Verilog Hardware Description Language (HDL) code for beginner’s who are learning digital circuits, however the same can be skipped by more advanced engineers who are already exposed to the fundamentals.

Some of the more advanced chapters like “*Design Guidelines for EMC performance*” have been thoroughly researched and have taken months to write in a way to make topics more relevant to digital designers.

Every possible effort was made to make the book self-contained. Any feedback/comments are welcome on this aspect or any other related aspects. Comments can be sent to me at the following mails: mohit.arora@me.com or mohit.arora@freescale.com.

Acknowledgements

The original idea behind “*The Art of Hardware Architecture*” was to link my years of experience as a design architect with the extensive research I have conducted. However, achieving the final shape of this book would not have been possible without many contributions.

My sweet wife *Pooja* was so patient with my late nights, and I want to thank her for her faithful support & encouragement in writing this book. Most of the work occurred on weekends, nights, while on vacation, and other times inconvenient to my family. I like to thank my parents for allowing me to follow my ambitions throughout my childhood.

I am grateful to *Prashant Bhargava*, my colleague from Freescale Semiconductor for his careful reading of drafts of this book and his constructive suggestions for improvement based on his years of experience. He also helped in book editing, formatting as well contribution to some of the sections in Chap. 7.

Special thanks to *Rakesh Pandey* and *Sudhi Ranjan Proch* from Freescale Semiconductor for their early help and feedback on some of the sections of the book.

I thank *Charles Glaser*, Senior Editor from Springer for providing me opportunity to publish with Springer and entire publication team at *Springer* who helped turn this book into excellent professional manuscript.

Mohit Arora

Contents

1	The World of Metastability	1
1.1	Introduction.....	1
1.2	Theory of Metastability.....	1
1.3	Metastability Window.....	3
1.4	Calculating MTBF	3
1.5	Avoiding Metastability.....	5
1.5.1	Using a Multi-stage Synchronizer	6
1.5.2	Multi-stage Synchronizer Using Clock Boost Circuitry.....	6
1.6	Metastability Test Circuitry	7
1.7	Types of Synchronizers.....	8
1.8	Metastability/General Recommendations.....	10
2	Clocks and Resets	11
2.1	Introduction.....	11
2.2	Synchronous Designs.....	11
2.2.1	Avoid Using Ripple Counters	12
2.2.2	Gated Clocks.....	12
2.2.3	Double-Edged or Mixed Edge Clocking.....	13
2.2.4	Flip Flops Driving Asynchronous Reset of Another Flop.....	13
2.3	Recommended Design Techniques	14
2.3.1	Avoid Combinational Loops in Design.....	14
2.3.2	Avoid Delay Chains in Digital Logic.....	16
2.3.3	Avoid Using Asynchronous Based Pulse Generator.....	16
2.3.4	Avoid Using Latches.....	17
2.3.5	Avoid Using Double-Edged Clocking	20
2.4	Clocking Schemes.....	22
2.4.1	Internally Generated Clocks	22
2.4.2	Divided Clocks.....	24
2.4.3	Ripple Counters	25
2.4.4	Multiplexed Clocks.....	25
2.4.5	Synchronous Clock Enables and Gated Clocks.....	26

2.5	Clock Gating Methodology.....	28
2.5.1	Latch Free Clock Gating Circuit.....	28
2.5.2	Latch Based Clock Gating Circuit.....	30
2.5.3	Gating Signals.....	32
2.5.4	Data Path Re-ordering to Reduce Switching Propagation.....	32
2.6	Reset Design Strategy.....	32
2.6.1	Design with Synchronous Reset.....	33
2.6.2	Design with Asynchronous Reset.....	36
2.6.3	Flip Flops with Asynchronous Reset and Asynchronous Set.....	38
2.6.4	Asynchronous Reset Removal Problem.....	40
2.6.5	Reset Synchronizer.....	40
2.6.6	Reset Glitch Filtering.....	42
2.7	Controlling Clock Skew.....	42
2.7.1	Short Path Problem.....	43
2.7.2	Clock Skew and Short Path Analysis.....	44
2.7.3	Minimizing Clock Skew.....	46
	References.....	49
3	Handling Multiple Clocks.....	51
3.1	Introduction.....	51
3.2	Multiple Clock Domains.....	51
3.3	Problems with Multiple Clock Domains Design.....	51
3.3.1	Setup Time and Hold Time Violation.....	53
3.3.2	Metastability.....	53
3.4	Design Tips for Efficient Handling of a Design with Multiple Clocks.....	54
3.4.1	Clock Nomenclature.....	54
3.4.2	Design Partitioning.....	55
3.4.3	Clock Domain Crossing.....	55
3.5	Synchronous Clock Domain Crossing.....	58
3.5.1	Clocks with the Same Frequency and Zero Phase Difference.....	59
3.5.2	Clocks with the Same Frequency and Constant Phase Difference.....	59
3.5.3	Clocks with the Different Frequency and Variable Phase Difference.....	60
3.6	Handshake Signaling Method.....	64
3.6.1	Requirements for Handshake Signaling.....	65
3.6.2	Disadvantages of Handshake Signaling.....	66
3.7	Data Transfer Using Synchronous FIFO.....	66
3.7.1	Synchronous FIFO Architecture.....	67

3.7.2	Working of Synchronous FIFO.....	68
3.8	Asynchronous FIFO (or Dual Clock FIFO).....	69
3.8.1	Avoid Using Binary Counters for the Pointer Implementation	70
3.8.2	Use Gray Coding Instead of Binary for the Counters.....	71
3.8.3	Gray Code Implementation of FIFO Pointers.....	74
3.8.4	FIFO Full and FIFO Empty Generation.....	79
3.8.5	Dual Clock FIFO Design	82
	References.....	86
4	Clock Dividers	87
4.1	Introduction.....	87
4.2	Synchronous Divide by Integer Value	87
4.3	Odd Integer Division with 50% Duty Cycle.....	88
4.4	Non-integer Division (with a Non 50% Duty Cycle)	90
4.4.1	Divide by 1.5 with Non 50% Duty Cycle	90
4.4.2	Counter Implementation for Divide by 4.5 (Non 50% Duty Cycle)	91
4.5	Alternate Approach for Divide by N.....	92
4.5.1	LUT Implementation for Divide by 1.5	93
	Reference	93
5	Low Power Design.....	95
5.1	Introduction.....	95
5.2	Sources of Power Consumption.....	95
5.3	Power Reduction at Different Levels of Design Abstraction.....	96
5.4	System Level Power Reduction	98
5.4.1	System on Chip (SoC) Approach.....	98
5.4.2	Hardware/Software Partitioning	98
5.4.3	Low Power Software.....	101
5.4.4	Choice of Processor	102
5.5	Architecture Level Power Reduction	102
5.5.1	Advanced Clock Gating	103
5.5.2	Dynamic Voltage and Frequency Scaling (DVFS)	104
5.5.3	Cache Based Architecture.....	105
5.5.4	Log FFT Architecture	106
5.5.5	Asynchronous (Clockless) Design.....	106
5.5.6	Power Gating.....	108
5.5.7	Multi-threshold Voltage	111
5.5.8	Multi-supply Voltage.....	112
5.5.9	Gate Memory Power	112
5.6	Register Transfer Level (RTL) Power Reduction	113
5.6.1	State Machine Encoding and Decomposition.....	113
5.6.2	Binary Number Representation.....	114

5.6.3	Basic Gated Clock.....	115
5.6.4	One Hot Encoded Multiplexer.....	117
5.6.5	Removing Redundant Transactions	118
5.6.6	Resource Sharing	119
5.6.7	Using Ripple Counters for Low Power.....	121
5.6.8	Bus Inversion	124
5.6.9	High Activity Nets	124
5.6.10	Enabling-Disabling Logic Clouds.....	125
5.7	Transistor Level Power Reduction.....	126
5.7.1	Technology Level.....	126
5.7.2	Layout Optimization	127
5.7.3	Substrate Biasing	127
5.7.4	Reduce Oxide Thickness.....	127
5.7.5	Multi-oxide Devices.....	127
5.7.6	Minimizing Capacitance by Custom Design	128
	References.....	128
6	The Art of Pipelining	129
6.1	Introduction.....	129
6.2	Factors Affecting the Maximum Frequency of Clock	129
6.2.1	Clock Skew	131
6.2.2	Clock Jitter.....	131
6.3	Pipelining.....	133
6.4	Pipelining Explained – A Real Life Example.....	136
6.5	Performance Increase from Pipelining	137
6.6	Implementation of DLX Instruction	140
6.7	Effect of Pipelining on Throughput	144
6.8	Pipelining Principles.....	145
6.9	Pipelining Hazards.....	146
6.9.1	Structural Hazards	146
6.9.2	Data Hazards.....	147
6.9.3	Control Hazards	150
6.9.4	Other Hazards	151
6.10	Pipelining in ADC – An Example	152
	References.....	153
7	Handling Endianness.....	155
7.1	Introduction.....	155
7.2	Definition	155
7.3	Little-Endian or Big-Endian: Which Is better?.....	157
7.4	Issues Dealing with Endianness Mismatch.....	158
7.5	Accessing 32 Bit Memory	160
7.6	Dealing with Endianness Mismatch	161

7.6.1	Preserve Data Integrity (Data Invariance).....	161
7.6.2	Address Invariance.....	163
7.6.3	Software Byte Swapping.....	166
7.7	Endian Neutral code.....	167
7.8	Endian-Neutral Coding Guidelines	167
	References.....	168
8	Debouncing Techniques	169
8.1	Introduction.....	169
8.2	Behavior of a Switch.....	170
8.3	Switch Types.....	171
8.4	De-bouncing Techniques	172
8.4.1	RC De-bouncer	172
8.4.2	Hardware De-bouncers	176
8.4.3	Software De-bouncing	177
8.4.4	De-bouncing Guidelines	179
8.4.5	De-bouncing on Multiple Inputs.....	180
8.5	Existing Solutions	181
9	Design Guidelines for EMC Performance	183
9.1	Introduction.....	183
9.2	Definition	183
9.3	EMI Theory and Relationship with Current and Frequency.....	185
9.4	EMI Regulations, Standards and Certification.....	186
9.5	Factors Affecting IC Immunity Performance	187
9.5.1	Microcontroller as Noise Source	187
9.5.2	Other Factors Affecting EMC.....	188
9.5.3	Noise Carriers	189
9.6	Techniques to Reduce EMC/EMI	189
9.6.1	System Level Techniques.....	190
9.6.2	Board Level Techniques.....	192
9.6.3	Microcontroller Level Techniques	201
9.6.4	Software Level Techniques	205
9.6.5	Other Techniques	212
9.7	Summary	213
	References.....	214
	References.....	215
	Index.....	219

Chapter 1

The World of Metastability

1.1 Introduction

In a synchronous system, the data always has a fixed relationship with respect to the clock. When that relationship obeys the setup and hold requirements for the device, the output goes to a valid state within its specified propagation delay time. In synchronous systems, the input signals always meet the flip-flop's timing requirements; therefore, metastability does not occur. However, in an asynchronous system, the relationship between data and clock is not fixed; therefore, occasional violations of setup and hold times can occur. When this happens, the output may go to an intermediate level between its two valid states and remain there for an indefinite amount of time before resolving itself or it may simply be delayed before making a normal transition.

This Chapter is intended to help understand more clearly the issues relating to the metastability, how it is quantified, and how to minimize its effort.

1.2 Theory of Metastability

Metastability arises as a result of violation of setup and hold times of a flip flop. Every flip-flop that is used in any design has a specified setup and hold time, or the time during which the data input is not legally permitted to change before and after a rising clock edge, respectively. If the signal does change during this time window, the output will be unknown or “metastable”. This propagation of unwanted state is called Metastability. As a result the output of a flip-flop can produce a glitch or remain temporarily in metastable state, thus taking longer to return to stable state.

When a flip-flop is in a metastable state, the output hovers at a voltage level between high and low, causing the output transition to be delayed beyond the specified clock-to-output delay (t_{co}). The additional time beyond t_{co} that a metastable output takes to resolve to a stable state is called the settling time (t_{MET}). This has

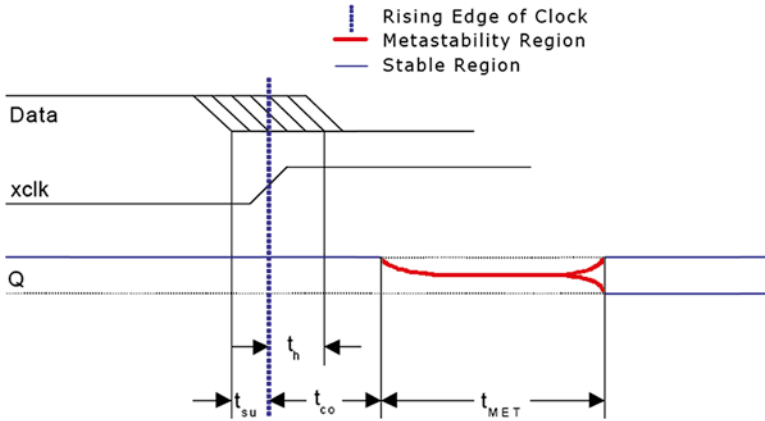


Fig. 1.1 Metastability timing parameters

been shown in Fig. 1.1. Not every transition that violates the setup or hold times results in a metastable output. The likelihood that a flip-flop enters a metastable state and the time required to return to stable state depends on the process technology used to manufacture the device and on the ambient conditions. Generally, flip-flops will return to a stable state within one or two clock cycles.

The operation of a Flip-Flop is analogous to a ball rolling over a frictionless hill, as shown in Fig. 1.2. Each side of the hill represents a stable state (i.e. high or low) and the top represents a metastable state. Suppose the ball is in a stable state (i.e. either 1 or 0) and a push (state transition) is given to the ball that is sufficient (no setup or hold time violations) enough to make the ball cross over to the other stable state, the ball crosses to the other stable state within the specified time.

However, if the push is less (i.e. violation of setup and hold time), the ball shall travel to the top of the hill (i.e. output metastable), stay there for some time and return to either stable state (i.e. output becomes stable eventually). It may also happen that the ball may rise partially and come back (i.e. output may produce some glitches). Either condition increases the delay from clock transition to a stable output.

Thus, in simple words, when a signal is changing in one clock domain (*src_data_out*) and is sampled in another clock domain (*dest_data_in*), then this causes the output to become metastable. This is known as Synchronization Failure (Shown in Fig. 1.3).

1.3 Metastability Window

Metastability Window is defined as the specific length of time, during which both the data and clock should not change. If both signals do occur, the output may go metastable. As shown in Fig. 1.4, the combination of the Setup and Hold time determine the width of the Metastability window.

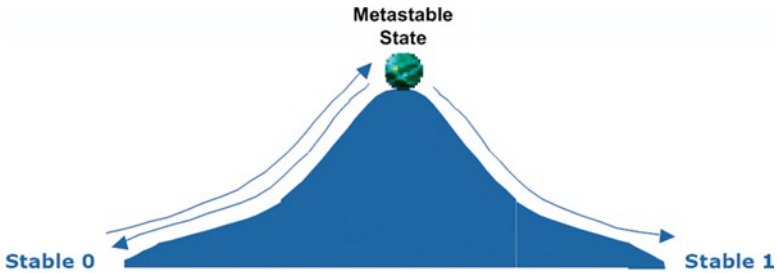


Fig. 1.2 Metastable behavior of flip flop

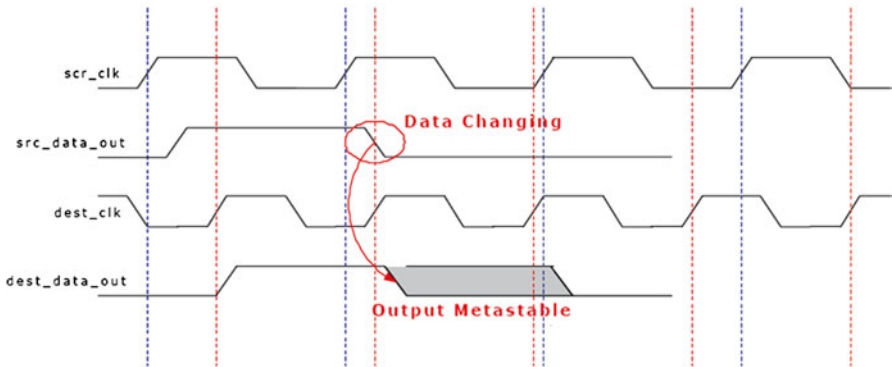


Fig. 1.3 Metastability in flip flop

The larger the window, the greater the chance the device will go Metastable. In most cases, newer logic families have smaller Metastability windows which reduce the chance of the device going Metastable.

1.4 Calculating MTBF

Mean (Average) Time Between Failures or MTBF of a system, is the reciprocal of the failure rate in the special case when the failure rate is constant. This gives the information on how often a particular Flip Flop will fail.

For a single-stage synchronizer with a given clock frequency and an asynchronous data edge that has a uniform probability density within the clock period, the rate of generation of metastable events can be calculated by taking the ratio of the setup and hold time window to the time between clock edges and multiplying by the data edge frequency.

Fig. 1.4 Metastability window

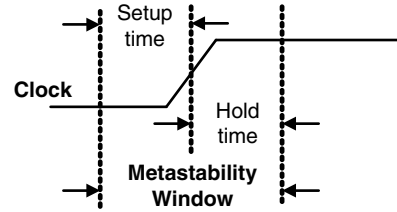
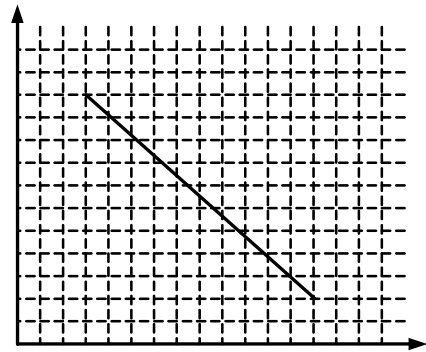


Fig. 1.5 Failure rate vs. time (log scale)



$$\frac{1}{\text{FailureRate}} = \text{MTBF}_1 = \frac{e(t_r / \tau)}{W \times f_c \times f_d} \quad (1.1)$$

where

t_r = resolve time allowed in excess of the normal propagation delay time of the device

τ = metastability (resolving) time constant for a flip-flop

W = Metastability Window

f_c = Clock frequency

f_d = Asynchronous data edge frequency

The constants W and τ are related to electrical characteristics of the device and may vary according to the process technology node. Therefore, different devices manufactured with the same process have similar values for W and τ .

If the failure rate of a device is measured at different resolve times and plotted, the result is an exponentially decaying curve. When plotted on a semi logarithmic scale, as shown in Fig. 1.5, this becomes a straight line the slope of which is equal to τ ; therefore, two data points on the line are sufficient to calculate the value of τ using Eq. 1.2.

$$\tau = \frac{t_{r2} - t_{r1}}{\ln(N1 / N2)} \quad (1.2)$$

where

t_{r1} = resolve time 1

t_{r2} = resolve time 2

$N1$ = numbers of failures at t_{r1}

$N2$ = numbers of failures at t_{r2}

Based on Eqs. 1.1 and 1.2, MTBF for a two-stage synchronizer can be calculated by Eq. 1.3 below

$$MTBF_2 = \frac{e(t_{r1} / \tau)}{W \times f_c \times f_d} \times e(t_{r2} / \tau) \quad (1.3)$$

where

t_{r1} = resolve time allowed for the first stage of synchronizer

t_{r2} = resolve time in access of normal propagation delay

The first term in the Eq. 1.3 calculates the MTBF of the first stage of the synchronizer, which in effect becomes the generation rate of the metastable events for the next stage. The second term then calculates the probability that the metastable event will be resolved based on the value of t_{r2} , the resolve time allowed external to the synchronizer. The product of the two terms gives the overall MTBF for the two-stage synchronizer.

In quantitative terms, using Eq. 1.3 above, if the Mean Time Between Failure (MTBF) of a particular Flip-Flop in the context of a given clock rate and the input transition rate is 40 s then MTBF of two such flip-flops used to synchronize the input would be $40 \times 40 = 26.6$ min.

1.5 Avoiding Metastability

As shown in Sect. 1.2, metastability occurs whenever setup or hold time is violated. So signals may violate the timing requirements under the following conditions:

- When the input signal is an asynchronous signal.
- When the clock skew/slew (rise/fall times) is higher than the tolerable limit.
- When signals cross the domains working at two different frequencies or with same frequency but different phase and skew.
- When the combinational delay is such that the Flip Flop data input changes in the Metastability Window.

Metastability can cause excessive propagation delays and subsequent system failures. All Flip Flops and latches exhibit metastability. The problem cannot be eliminated. But it is possible to make metastability less likely to occur.

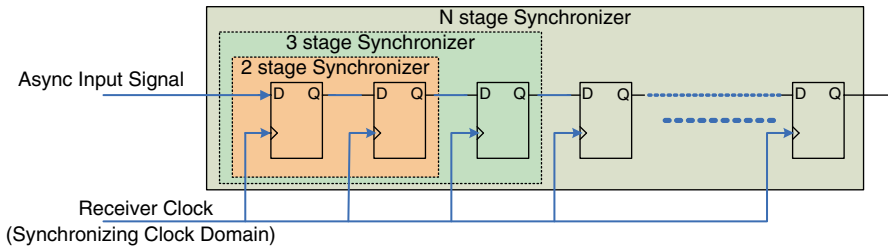


Fig. 1.6 N-stage synchronizer

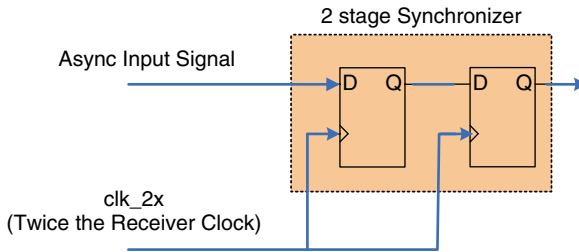


Fig. 1.7 Multi-stage synchronizer with clock boost circuitry

In the simplest case, designers can avoid metastability by making sure the clock period is long enough to allow for the resolution of quasi-stable states and for the delay of whatever logic may be in the path to the next flip-flop. This approach, while simple, is rarely practical given the performance requirements of most modern designs. The other approach is to use Synchronizers.

1.5.1 Using a Multi-stage Synchronizer

The most common way to avoid metastability is to add one or more synchronizing flip-flops at the signals that move from one clock domain to the other as shown in Fig. 1.6. This approach allows for an entire clock period (except for the setup time of the second flip-flop) for metastable events in the first synchronizing flip-flop to resolve itself. This does however; increase the latency in the synchronous logic's observation of input.

1.5.2 Multi-stage Synchronizer Using Clock Boost Circuitry

One limitation of the multiple-stage synchronizer is that it takes longer for the system to respond to an asynchronous input. A solution to this problem is to use the output of a clock doubler to clock the two synchronizing flip-flops. Altera's FPGA exhibit this technique as Clock Boost or Clock Doubler (Fig. 1.7).

This approach allows the system to respond to an asynchronous input within one system clock cycle, while still improving MTBF. Although the Clock Boost clock could decrease the MTBF, this effect is more than offset by the two synchronizing flip-flops.

Neither of these approaches can guarantee that metastability cannot pass through the synchronizer; they simply reduce the probability of occurrence of metastability.

1.6 Metastability Test Circuitry

Whenever a flip-flop samples an asynchronous input, a small probability exists that the flip-flop output will exhibit an unpredictable delay. This happens not only when the input transition violates setup and hold time specifications, but also when the transition actually occurs within a small timing window during which the flip-flop accepts the new input. Under these circumstances, the flip-flop can enter a metastable state.

The test circuit described in Fig. 1.8 is used to determine metastability characteristics of a Flip-Flop. Figure 1.8 shows an Asynchronous Input “*async_In*” to the Flip Flop “FF_A” triggered on positive edge of Clock “*clk*”. As shown both the Flops “FF_B” and “FF_C” are triggered on negative edge of the clock in order to capture the metastable event on “FF_A”.

As complementary signals are passed on the input of Flip Flops “FF_B” and “FF_C”, the output of the XNOR gate goes HIGH whenever a metastable event occurs on “FF_A”. This conditions is captured on output of Flip Flop “FF_D” indicating that a metastable event has been detected.

The timing for all the nodes in this test circuit is shown in Fig. 1.9.

Because the resolving flip-flops (“FF_B” and “FF_C”) are clocked by the falling clock edge, the required settling time can be controlled by changing the clock high time (Δt). The settling time t_{MET} can be determined with the equation below

$$t_{MET} = \Delta t - t_{ACN} \quad (1.4)$$

where t_{ACN} is the minimum clock period which is equal to t_{CQ} (clock to Output delay of FF_A) + setup time t_{su} of the resolving Flip Flop (FF_B or FF_C).

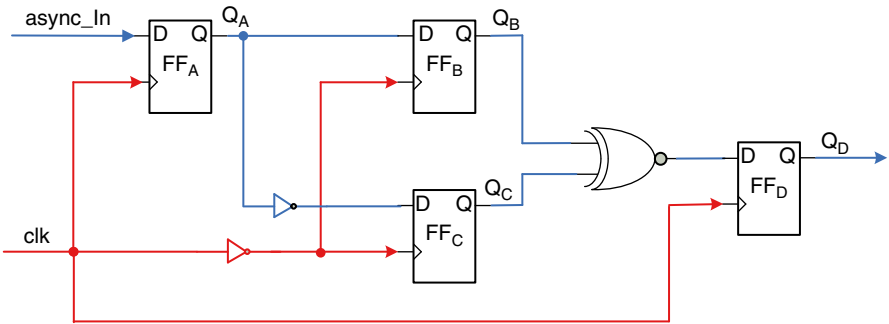


Fig. 1.8 Metastability test circuitry

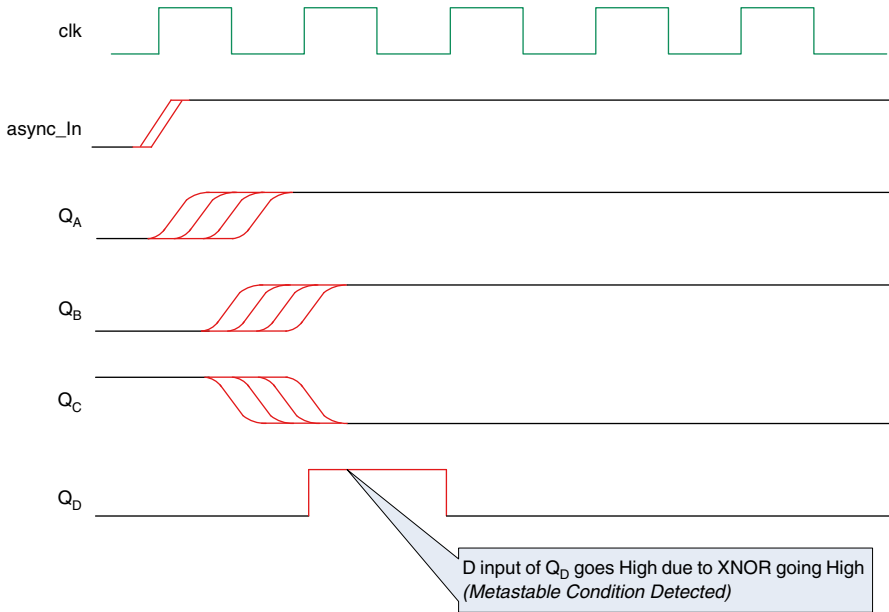


Fig. 1.9 Timings for metastability test circuitry

One of the methods to reduce the resolve time or settling time is to add jitter to the data centered across setup/hold.

1.7 Types of Synchronizers

As per Eq. 1.1, Mean Time Between Failures (MTBF) of a circuit with an asynchronous input is exponentially related to the time available for recovery from a metastable condition. Use synchronizers to create a time buffer for recovering from a metastable event.

Note that an asynchronous signal should never be synchronized by more than one synchronizer. (To do so would risk having the outputs of multiple synchronizers produce different signals). This section shows two-synchronizer schemes A and B.

Scheme A is normal scheme and works best when the width of the Asynchronous Input signal is greater than the clock period (Fig. 1.10).

Note that even if the asynchronous input reaches a stable condition outside the setup interval, it will still be clocked through with a latency of two clock cycles otherwise FF1 may enter metastability.

If metastability is resolved in less than one clock cycle, FF2 will have a stable input else deeper cascading is required as shown in Fig. 1.6.

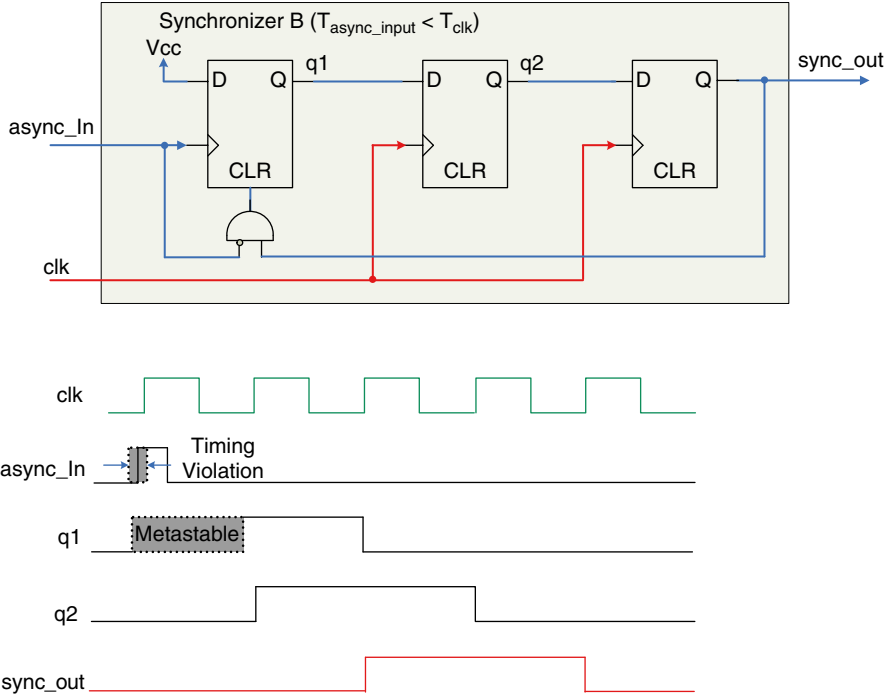


Fig. 1.10 Synchronizer Scheme A for a two-stage synchronizer

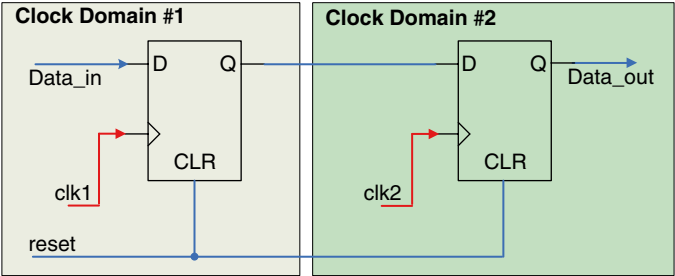


Fig. 1.11 Synchronizer Scheme B for a two-stage synchronizer

However, Scheme A does not work where the width of the Asynchronous Input is smaller than the clock period. In this case, scheme B works the best as shown in Fig. 1.11.

Note that incase of Synchronizer scheme B, D input of the first FF (Flip Flop) is connected to V_{CC} , while asynchronous input clocks the FF. The other two FF in the stage are clocked directly by system clock or *clk*. A short pulse will drive *q1* High that will propagate to *sync_out* after two “*clk*” edges.

So this defines our rule of thumb that is summarized as follows

1. Use synchronizers when a signal must cross a boundary between clock domains.
2. If $\text{Clk1} < \text{Clk2}$ use Synchronizer scheme A at the input of the clock domain 2 (as shown in Fig. 1.11) , otherwise use synchronizer scheme B.

1.8 Metastability/General Recommendations

Metastability cannot be avoided at the boundary between two systems that are asynchronous with respect to each other. However the probability that metastable states are encountered can be significantly reduced by the following recommendations:

- (a) Use Synchronizers.
- (b) Use Faster Flip Flops (narrower metastable window T_w).
- (c) Use metastable hardened Flip Flops (designed for very high bandwidth and reduced sampling times that are optimized for clock domain input circuitry).
- (d) Cascade flip-flops as Synchronizers (two or more) as shown in Fig. 1.6. A chain of N Flip Flops has a probability of P^N where P is the chance of metastable failure for one flip flop.
- (e) Reduce Sampling rate.
- (f) Avoid input signals with low dV/dt .

Chapter 2

Clocks and Resets

2.1 Introduction

The cost of designing ASICs is increasing every year. In addition to the non-recurring engineering (NRE) and mask costs, development costs are increasing due to ASIC design complexity. To overcome the risk of re-spins, high NRE costs, and to reduce time-to-market delays, it has become very important to design the first time working silicon.

This chapter constitutes a general set of recommendations intended for use by designers while designing a block or an IP (Intellectual Property). The guidelines are independent of any CAD tool or silicon process and are applicable to any ASIC designs and can help designers to plan and to execute a successful System on Chip (SoC) with a well-structured and synthesizable RTL code.

The current paradigm shift towards system level integration (SLI), incorporating multiple complex functional blocks and a variety of memories on a single circuit, gives rise to a new set of design requirements at integration level. The recommendations are principally aimed at the design of the blocks and memory interfaces which are to be integrated into the system-on-chip. However, the guidelines given here are fully consistent with the requirements of system level integration and will significantly ease the integration effort, and ensure that the individual blocks are easily reusable in other systems.

These guidelines can form as a basis of checklist that can be used as a signoff for each design prior to submission for fabrication.

2.2 Synchronous Designs

Synchronous designs are characterized by a single master clock and a single master set/reset driving all sequential elements in the design.

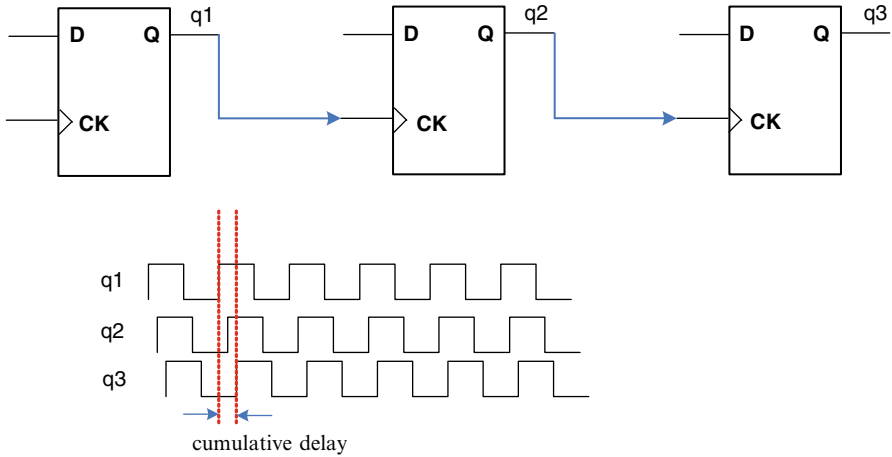


Fig. 2.1 Flip flop driving the clock input of another flip flop (ripple counter)

Experience has shown that the safest methodology for time domain control of an ASIC is synchronous design. Some of the problems with the circuits not being synchronous have been shown in this section.

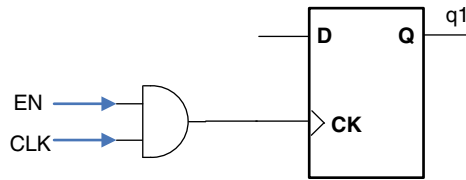
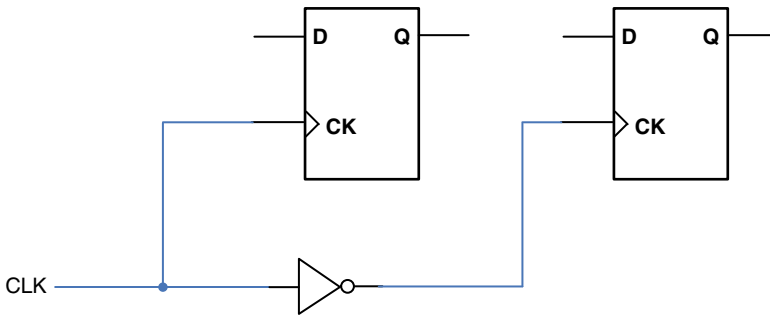
2.2.1 Avoid Using Ripple Counters

Flip Flops driving the clock input of other flip flops is somewhat problematic. The clock input of the second flip-flop is skewed by the *clock-to-q* delay of the first flip-flop, and is not activated on every clock edge. This cumulative effect with more than two Flip Flops connected in a similar manner forms a Ripple counter as shown in Fig. 2.1. Note the cumulative delay gets added on with more number of flip flops and hence the same is not recommended. More details on the ripple counter are given in Sect. 5.6.7.

2.2.2 Gated Clocks

Gating in a clock line causes clock skew and can introduce spikes which trigger the flip-flop. This is particularly the case when there is a multiplexer in the clock line as shown in Fig. 2.2.

Simulating a gated clock design might work perfectly fine but the problem arises when such a design is synthesized.

**Fig. 2.2** Gated clock line**Fig. 2.3** Double-edged clocking

2.2.3 Double-Edged or Mixed Edge Clocking

As shown in Fig. 2.3, the two flip-flops are clocked on opposite edges of the clock signal. This makes synchronous resetting and test methodologies such as scan-path insertion difficult, and causes difficulties in determining critical signal paths.

2.2.4 Flip Flops Driving Asynchronous Reset of Another Flop

In Fig. 2.4, the second flip-flop can change state at a time other than the active clock edge, violating the principle of synchronous design. In addition, this circuit contains a potential race condition between the clock and reset of the second flip-flop.

The subsequent sections show the methods to avoid the above non-recommended circuits.

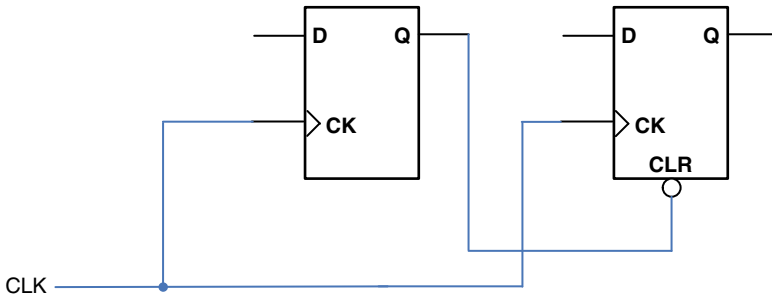


Fig. 2.4 Flip flop driving asynchronous reset of another flop

2.3 Recommended Design Techniques

When designing with HDL code, it is important to understand how a synthesis tool interprets different HDL coding styles and the results to expect. It is very important to think in terms of hardware as a particular design style (or rather coding style) can affect gate count and timing performance. This section discusses some of the basic techniques to ensure optimal synthesis results while avoiding several causes of unreliability and instability.

2.3.1 Avoid Combinational Loops in Design

Combinational loops are among the most common causes of instability and unreliability in digital designs. In a synchronous design, all feedback loops should include registers. Combinational loops violate synchronous design principles by establishing a direct feedback with no registers.

In terms of HDL language, combinational loops occur when the generation of a signal depends on itself through several combinational *always*¹ blocks or when the left-hand side of an arithmetic expression also appears on the right-hand side. Combo loops are a hazard to a design and synthesis tools will always give errors when combo loops are encountered, as these are not synthesizable.

The generation of combo loops can be understood from the following bubble diagram in Fig. 2.5. Each bubble represents a combo *always* block and the arrow going into it represents the signal being used in that *always* block while an arrow going out from the bubble represents the output signal generated by that output block. It is evident that the generation of signal 'a' depends on itself through signal 'd', thereby generation a combinational loop.

¹For simplicity, any HDL languages that this book refers to takes Verilog as an example.

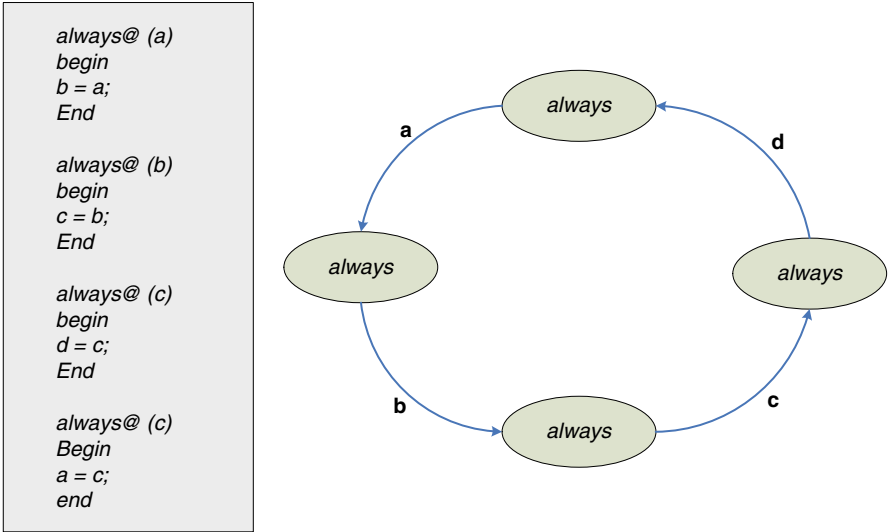


Fig. 2.5 Combinational loop example and bubble diagram

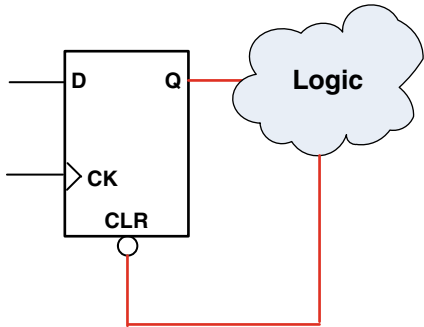


Fig. 2.6 Combinational loop through asynchronous control pins

The code and the bubble diagram are shown below [28]:

In order to remove combo loops, one must change the generation of one of the signals so the dependency of signals on each other is removed. Simple resolution to this problem is to introduce a Flip Flop or register in the combo loop to break this direct path.

Figure 2.6 shows another example where output of a register directly controls the asynchronous pin of the same register through combinational logic.

Combinational loops are inherently high-risk design structures. Combinational loop behavior generally depends on the relative propagation delays through the logic involved in the loop. Propagation delays can change based on various factors and the behavior of the loop may change. Combinational loops can cause endless

computation loops in many design tools. Most synthesis tools break open or disable combinatorial loop paths in order to proceed. The various tools used in the design flow may open a given loop a different manner, processing it in a way that may not be consistent with the original design intent.

2.3.2 Avoid Delay Chains in Digital Logic

Delay chains occur when two or more consecutive nodes with a single fan-in and a single fan-out are used to cause delay. Often inverters are chained together to add delay. Delay chains generally result from asynchronous design practices, and are sometimes used to resolve race conditions created by other combinational logic. In both FPGA and ASIC, delays can change with each place-and-route. Delay chains can cause various design problems, including an increase in a design's sensitivity to operating conditions, a decrease in a design's reliability, and difficulties when migrating to different device architecture. Avoid using delay chains in a design, rely on synchronous practices instead.

2.3.3 Avoid Using Asynchronous Based Pulse Generator

Often design requires generating a pulse based on some events. Designers sometimes use delay chains to generate either one pulse (pulse generators) or a series of pulses (multi-vibrators). There are two common methods for pulse generation; these techniques are purely asynchronous and should be avoided:

- A trigger signal feeds both inputs of a two-input AND or OR gate, but the design inverts or adds a delay chain to one of the inputs. The width of the pulse depends on the relative delays of the path that feeds the gate directly and the one that goes through the delay. This is the same mechanism responsible for the generation of glitches in combinational logic following a change of inputs. This technique artificially increases the width of the spike by using a delay chain.
- A register's output drives the same register's asynchronous reset signal through a delay chain. The register essentially resets itself asynchronously after a certain delay.

Asynchronously generated pulse widths often pose problem to the synthesis and place-and-route software. The actual pulse width can only be determined when routing and propagation delays are known, after placement and routing. So it is difficult to reliably determine the width of the pulse when creating HDL. The pulse may not be wide enough for the application in all PVT conditions, and the pulse width will change when migrating to a different technology node. In addition, static timing analysis cannot be used to verify the pulse width so verification is very difficult.

Multi-vibrators use the principle of the "glitch generator" to create pulses, in addition to a combinational loop that turns the circuit into an oscillator [25].

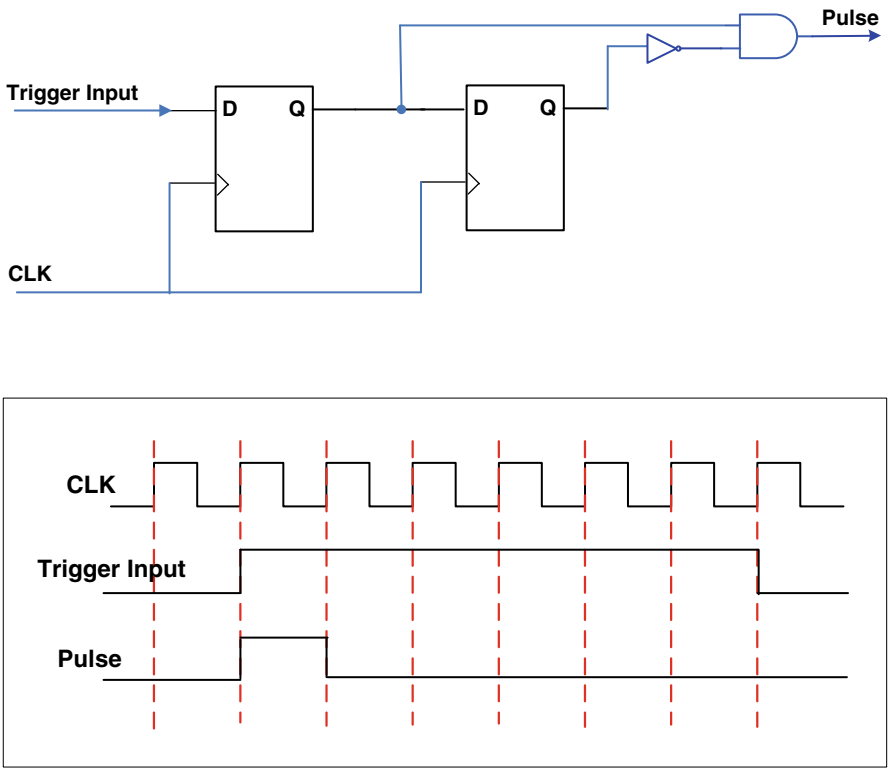


Fig. 2.7 Synchronous pulse generator circuit on start of trigger input

Structures that generate multiple pulses cause even more problems than pulse generators because of the number of pulses involved. In addition, when the structures generate multiples pulses, they also increase the frequency of the design.

A recommended Synchronous Pulse generator is shown in Fig. 2.7.

In the above synchronous pulse generator design, the pulse width is always equal to the clock period. This pulse generator is predictable, can be verified with timing analysis, and is easily migrated to other architectures and is technology independent.

Similar to Fig. 2.7, Fig. 2.8 shows the pulse generator at the end of trigger input.

2.3.4 Avoid Using Latches

In digital logic, latches hold the value of a signal until a new value is assigned. Latches should be avoided whereas possible in the design and flip-flops should be used instead.

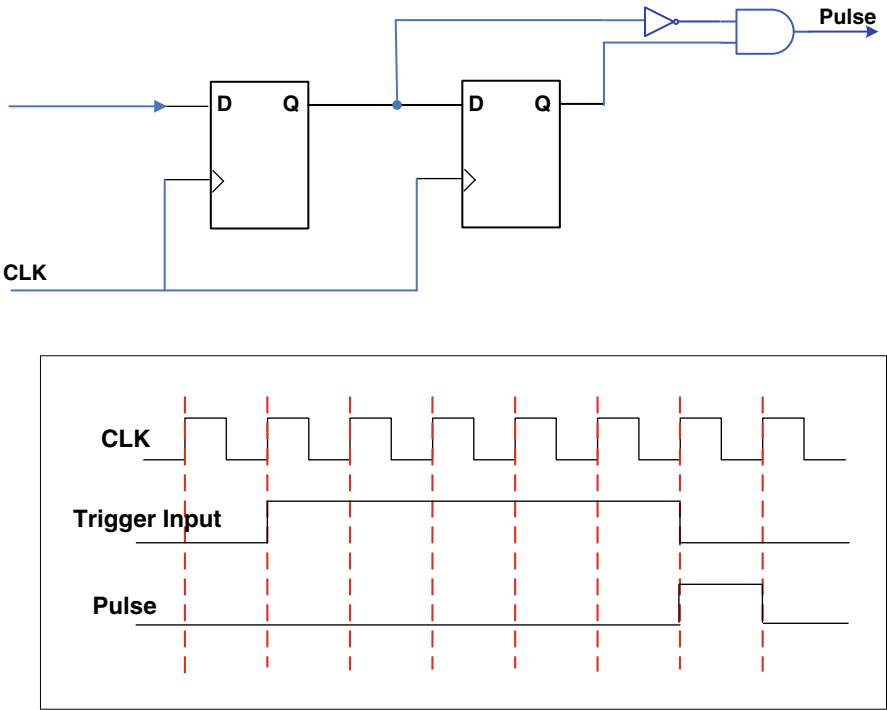


Fig. 2.8 Synchronous pulse generator circuit on end of trigger input

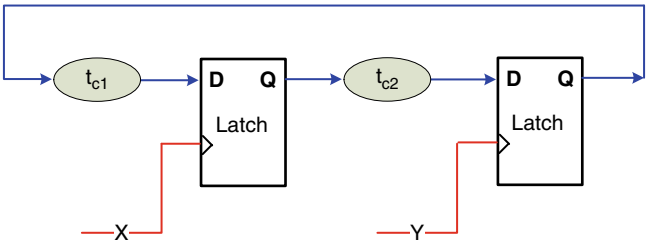


Fig. 2.9 Race conditions in latches

As shown in Fig. 2.9, if both the X and Y were to go high, and since these are level triggered, both the Latches would be enabled resulting in the circuit to oscillate.

Latches can cause various difficulties in the design. Although latches are memory elements like registers, they are fundamentally different. When a latch is in a feed-through mode, there is a direct path between the data input and the output. Glitches on the data input can pass to the output.

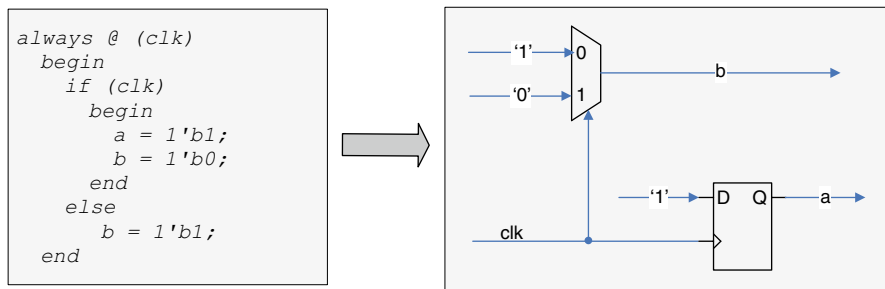


Fig. 2.10 Inferred latch due to incomplete 'if else' statement

Static timing analyzers typically make incorrect assumptions about latch transparency, and either find a false timing path through the input data pin or miss a critical path altogether. The timing for latches is also inherently ambiguous. When analyzing a design with a D latch, for example, the tool cannot determine whether you intended to transfer data to the output on the leading edge of the clock or on the trailing edge. In many cases, only the original designer knows the full intent of the design, which implies that another designer cannot easily migrate the same design or reuse the code.

Latches tend to make circuits less testable. Most design for test (DFT) and automatic test program generator (ATPG) tools do not handle latches very well.

Latches pose different challenge in FPGA designs as FPGA's are register-intensive; therefore, designing with latches uses more logic and leads to lower performance than designing with registers.

Synthesis tools occasionally infer a latch in a design when one is not intended. Inferred latches typically result from incomplete "if" or "case" statements. Omitting the final "else" clause in an "if" or "case" statement can also generate a latch. Figure 2.10 shows a similar example.

As shown in Fig. 2.10, 'b' will be synthesized as straight combinational logic while a latch will be inferred on signal 'a'.

A general rule for latch inferring is that if a variable is not assigned in all possible executions of an always statement (for example, when a variable is not assigned in all branches of an 'if' statement), then a latch is inferred.

Some FPGA architectures do not support latches. When such a design is synthesized, the synthesis tool creates a combinational feedback loop instead of a latch (as shown in Fig. 2.11).

Combinational feedback loops as shown above are capable of latching data but pose more problem than latches since they may violate setup, hold requirements which are difficult to be determined, whereas latches do not have any setup time, hold time violations since they are level triggered.

Note: The design should not contain any combinational feedback loops. They should be replaced by flip-flops or latches or be eliminated by fully enumerating RTL conditionals.

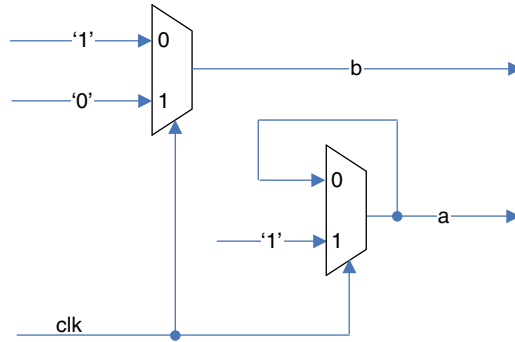


Fig. 2.11 Combinational loop implemented due to incomplete ‘if else’ statement

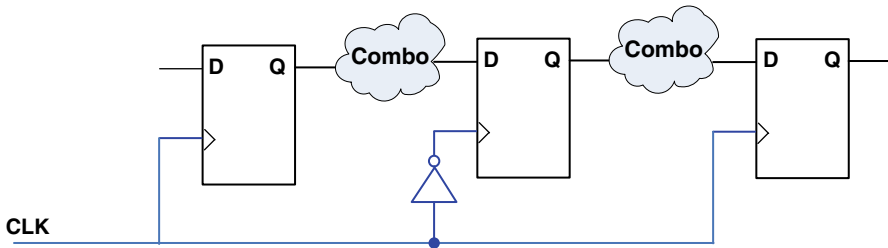


Fig. 2.12 Logic with double edged clocking

To conclude, this does not mean latches should never exist, we will see later how latches could be wonderful when it comes to cycle stealing or time borrowing to meet a critical path in a design.

2.3.5 Avoid Using Double-Edged Clocking

Double or Dual edged clocking is the method of data transfer on both the rising and falling edges of the clock, instead of just one or the other. The change allows for double the data throughput for a given clock speed.

Double edge output stage clocking is a useful way of increasing the maximum possible output speed from a design; however this violates the principle of Synchronous circuits and causes a number of problems.

Figure 2.12 shows a circuit triggered by both edges of clock.

Some of the problems encountered with Double Edged clocking are mentioned below:

- An asymmetrical clock duty cycle can cause setup and hold violations.
- It is difficult to determine critical signal paths.

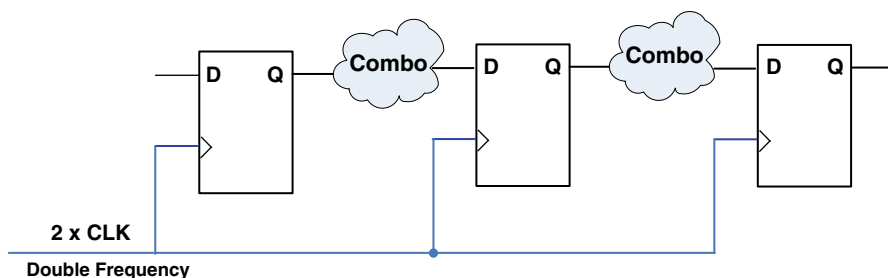


Fig. 2.13 Logic with single edged clocking

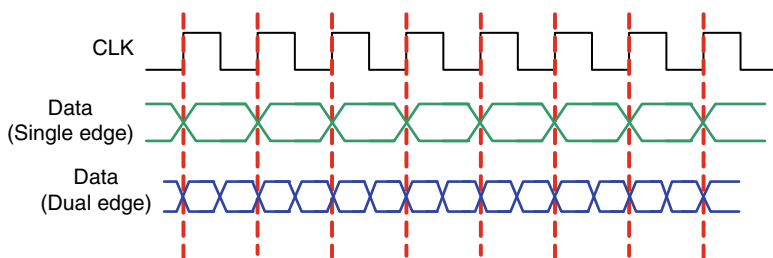


Fig. 2.14 Single/double edged data transfer

- Test methodologies such as scan-path insertion are difficult, as they rely on all flip-flops being activated on the same clock edge. If scan insertion is required in a circuit with double-edged clocking, multiplexers must be inserted in the clock lines to change to single-edged clocking in test mode.

Figure 2.13 shows the normal equivalent pipelined logic with single edge clocking. Note that this synchronous circuit requires a clock frequency that is double the one shown in Fig. 2.12.

Figure 2.14 shows the single transition and double transition clocked data transfer.

The green and blue signals represent data; the “hexagon” shapes are the traditional way of representing a signal that at any given time can be either a one or a zero.

In the circuit shown in Fig. 2.12, an asymmetrical clock duty cycle could cause setup and hold time violations, and a scan-path cannot easily be threaded through the flip-flops.

The above does not mean that circuits with dual edge clocking should never be used unless there is an intense desire for higher performance/speed that cannot be met with the equivalent synchronous circuits as the latter comes with an additional overhead of complexity in DFT and verification.

2.3.5.1 Advantages of Dual Edge Clocking

The one constant in the PC world is the desire for increased performance. This in turn means that most interfaces are, over time, modified to allow for faster clocking, which leads to improved throughput. Many newer technologies in the PC world have gone a step beyond just running the clock faster. They have also changed the overall signaling method of the interface or bus, so that data transfer occurs not once per clock cycle, but twice or more.

There are other advantages of circuit operating on dual edge rather than the same synchronous circuit being fed with double the clock frequency. Whatever extent possible, interface designers do regularly increase the speed of the system clock. However, as clock speeds get very high, problems are introduced on many interfaces. Most of these issues are related to the electrical characteristics of the signals themselves. Interference between signals increases with frequency and timing becomes more “tight”, increasing cost as the interface circuits must be made more precise to deal with the higher speeds.

The other advantage using double edged clocking is lower power consumptions as clock speeds are decreased by half and hence the system consumes less power than the equivalent synchronous circuits.

So to conclude system integrator should only use dual or double edged clocking unless the same desired performance cannot be met with the equivalent synchronous circuits.

2.4 Clocking Schemes

2.4.1 Internally Generated Clocks

A designer should avoid internally generated clocks, wherever possible, as they can cause functional and timing problems in the design, if not handled properly.

Clocks generated with combinational logic can introduce glitches that create functional problems and the delay due to the combinational logic can lead to timing problems. In a synchronous design, a glitch on the data inputs does not cause any issues and is automatically avoided as data is always captured on the edge of the clock and thus blocks the glitch. However, a glitch or a spike on the clock input (or an asynchronous input of a register) can have significant consequences.

Narrow glitches can violate the register’s minimum pulse width requirements. Setup and hold times may also be violated if the data input of the register is changing when a glitch reaches the clock input. Even if the design does not violate timing requirements, the register output can change value unexpectedly and cause functional hazards elsewhere in the design.

Figure 2.15 shows the effect of using a combinational logic to generate a clock on a synchronous counter. As shown in the timing diagram, due to the glitch on the clock edge, the counters increments twice in the clock cycle shown.

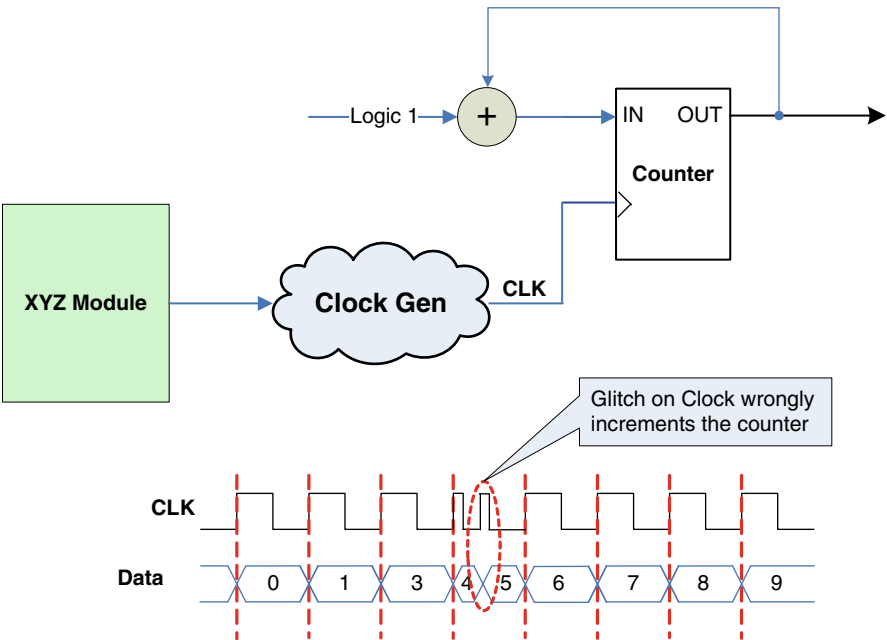


Fig. 2.15 Counter example for using combinational logic as a clock

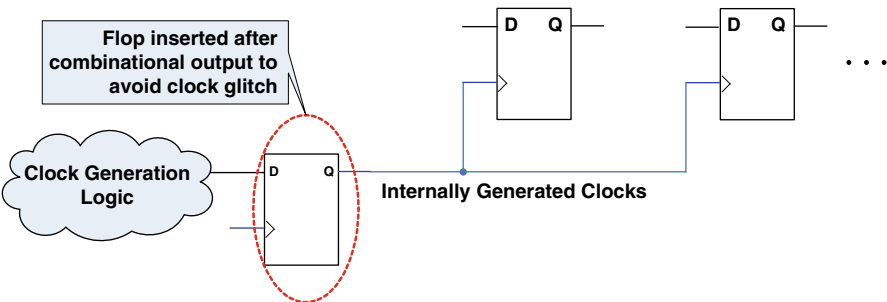


Fig. 2.16 Recommended clock generation technique

This extra counting may create functional issues in the design where instead of counting the desired count, counter counts an additional count due to the glitch on the clock.

Note: That for the sake of simplicity, it is assumed that the Counters Flops did not violate the setup/hold requirements on the data due to the glitch.

A simple guideline to the above problem is to always use a registered output of the combinational logic before using it as a clock signal. This registering ensures that the glitches generated by the combinational logic are blocked on the data input of the register (Fig. 2.16).

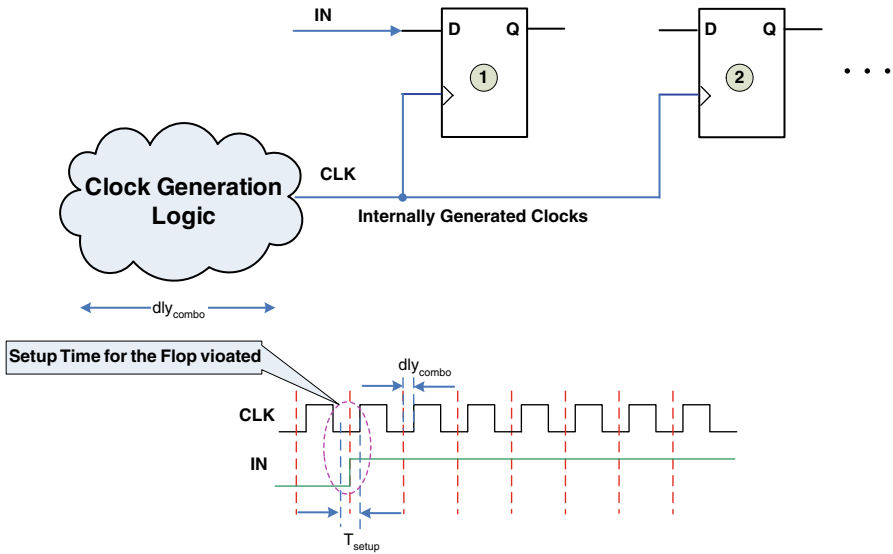


Fig. 2.17 Setup time violated due to skew of clock path

The combinational logic used to generate an internal clock also adds delays on the clock line. In some cases, logic delay on a clock line can result in a clock skew greater than the data path length between two registers. If the clock skew is greater than the data delay, the timing parameters of the register will be violated and the design will not function correctly.

Figure 2.17 shows a similar example where setup time on input “IN” is violated due to skew on the clock path.

Note: Data path delay is assumed to be zero for simplicity.

One solution to reduce the clock skew within the clock domain is by assigning the generated clock signal to one of the high-fanout and low-skew clock trees in the SoC. Using a low-skew clock tree can help reduce the overall clock skew for the signal.

2.4.2 Divided Clocks

Many designs require clocks created by dividing a master clock. Design should ensure that most of the clocks should come from the PLL. Using PLL circuitry will avoid many of the problems that can be introduced by asynchronous clock division logic. When using logic to divide a master clock, always use synchronous counters or state machines.

In addition, the design should ensure that registers always directly generate divided clock signals. Design should never decode the outputs of a counter or a state machine to generate clock signals; this type of implementation often causes glitches and spikes.

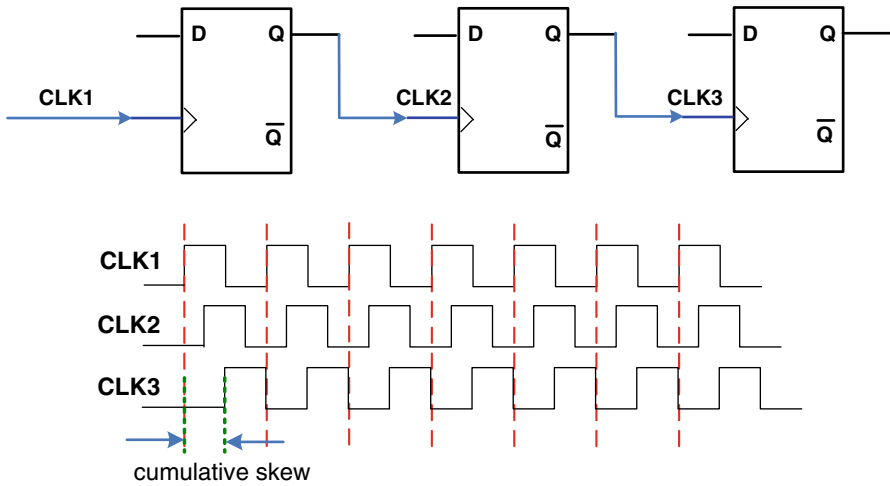


Fig. 2.18 Cascading effort in ripple counters

2.4.3 Ripple Counters

ASIC designers have often implemented ripple counters to divide clocks by a power of 2 because the counters use fewer gates than their synchronous counterparts. Ripple counters use cascaded registers, in which the output pin of each register feeds the clock pin of the register in the next stage (Fig. 2.18).

This cascading can cause problems because the counter creates a ripple clock at each stage. These ripple clocks pose another set of challenges for STA and synthesis tools. One should try to avoid these types of structures to ease verification effort.

Despite of all the challenges and problems with respect to using Ripple counters, these are quite handy in systems that eat power and can be good to reduce the peak power consumed by a logic or SoC.

Note: *Digital designers should consider using this technique in limited cases and under tight control.*

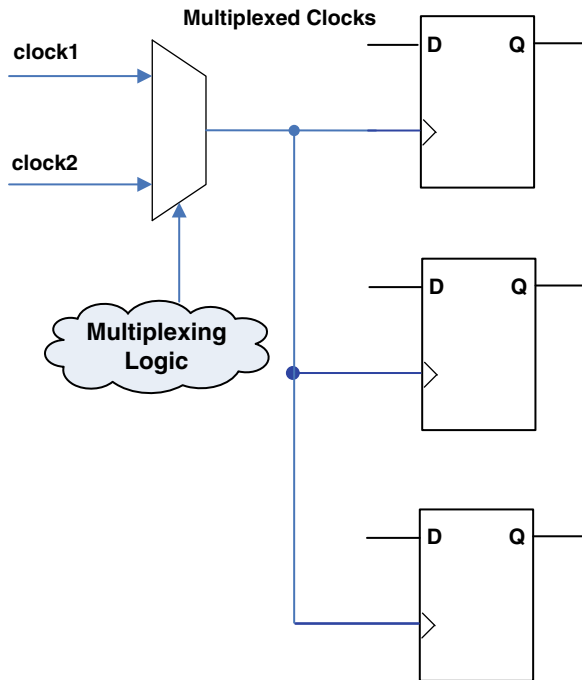
Refer Chap. 5 “Low power design” on more details analysis and techniques of using Ripple counters to save power consumption.

2.4.4 Multiplexed Clocks

Clock multiplexing can be used to operate the same logic function with different clock sources. Multiplexing logic of some kind selects a clock source as shown in Fig. 2.19.

For example, telecommunications applications that deal with multiple frequency standards often use multiplexed clocks.

Fig. 2.19 Multiplexing logic and clock sources



Adding multiplexing logic to the clock signal can lead to some of the problems discussed in the previous sections, but requirements for multiplexed clocks vary widely depending on the application.

Clock multiplexing is acceptable if the following criteria are met:

- The clock multiplexing logic does not change after initial configuration
- The design bypasses functional clock multiplexing logic to select a common clock for testing purposes
- Registers are always in reset when the clock switches
- A temporarily incorrect response following clock switching has no negative consequences

If the design switches clocks on the fly with no reset and the design cannot tolerate a temporarily incorrect response of the chip, then one must use a synchronous design so that there are no timing violations on the registers, no glitches on clock signals, and no race conditions or other logical problems.

2.4.5 Synchronous Clock Enables and Gated Clocks

Gated clocks turn a clock signal on and off using an enable signal that controls some sort of gating circuitry. As shown in Fig. 2.20, when a clock is turned off, the corresponding clock domain is shut down and becomes functionally inactive.

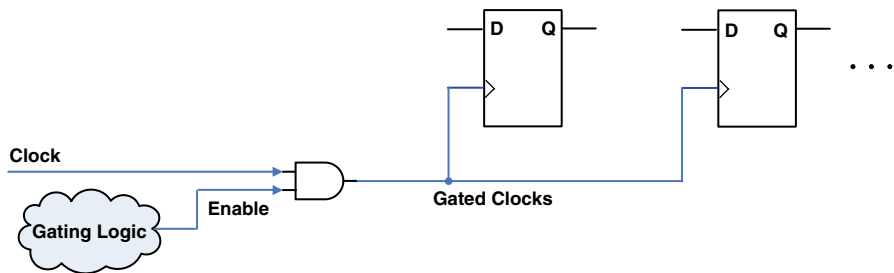


Fig. 2.20 Gated clock

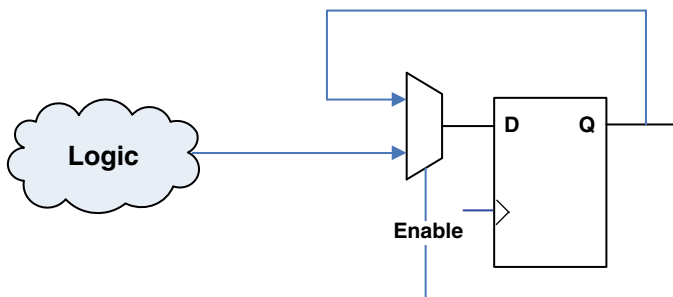


Fig. 2.21 Synchronous clock enable

Gated clocks can be a powerful technique to reduce power consumption. When a clock is gated both the clock network and the registers driven by it stop toggling, thereby eliminating their contributions to power consumption. However, gated clocks are not part of a synchronous scheme and therefore can significantly increase the effort required for design implementation and verification. Gated clocks contribute to clock skew and are also sensitive to glitches, which can cause design failure.

A clock domain can be turned off in a purely synchronous manner using a synchronous clock enable. However, when using a synchronous clock enable scheme, the clock tree keeps toggling and the internal circuitry of each Flip Flop remains active (although outputs do not change values), which does not reduce power consumption. A synchronous clock enable technique is shown in Fig. 2.21.

This Synchronous Clock Enable Clocking scheme does not reduce power consumption as much as gating the clock at the source because the clock network keeps toggling, but it will perform the same function as a gated clock by disabling a set of Flip Flops. As shown in Fig. 2.21, multiplexer in front of the data input of every Flip Flop either load new data or copy the output of the Flip Flop based on the Enable signal.

The next section is dedicated to efficient clock gating methodology that should be used where ever clocking gating is desired due to tight power specifications.

2.5 Clock Gating Methodology

In the traditional synchronous design style, the system clock is connected to the clock pin on every flip-flop in the design. This results in three major components of power consumption:

1. Power consumed by combinatorial logic whose values are changing on each clock edge (due to flops driving those combo cells).
2. Power consumed by flip-flops (this has non-zero value even if the inputs to the flip-flops, and therefore, the internal state of the flip-flops, is not changing).
3. Power consumed by the clock tree buffers in the design.

Gating the clock path substantially reduces the power consumed by a Flip Flop. Clock Gating can be done at the root of the clock tree, at the leaves, or somewhere in between.

Since the clock tree constitutes almost 50% of the whole chip power, it is always a good idea to generate and gate the clock at the root so that entire clock tree can be shut down instead of implementing the gating along the clock tree at the leaves.

Figure 2.22 shows an example of a clock gating for a three bit Counter.

The circuit is similar to the traditional implementation except that a clock gating element has been inserted into the clock network, which causes the flip-flops to be clocked only when the *INC* input is high. When the *INC* input is low, the flip-flops are not clocked and therefore retain the old data. This saves three multiplexers in front of the flip-flops which would have been there in case the gating was implemented by Synchronous Clock Enable as described in Fig. 2.21. This can result in significant area saving when wide banks of registers are being implemented.

2.5.1 Latch Free Clock Gating Circuit

The latch-free clock gating style uses a simple AND or OR gate (depending on the edge on which flip-flops are triggered) as shown in Fig. 2.23.

For the correct operation the circuit imposes a requirement that all enable signals be held constant from the active (rising) edge of the clock until the inactive (falling) edge of the clock to avoid truncating the generated clock pulse prematurely or generating multiple clock pulses (or glitches in clock) where one is required.

Figure 2.24 shows the case where generated clock is truncated prematurely when the above requirement is not satisfied.

This restriction makes the latch-free clock gating style inappropriate for our single-clock flip-flop based design.

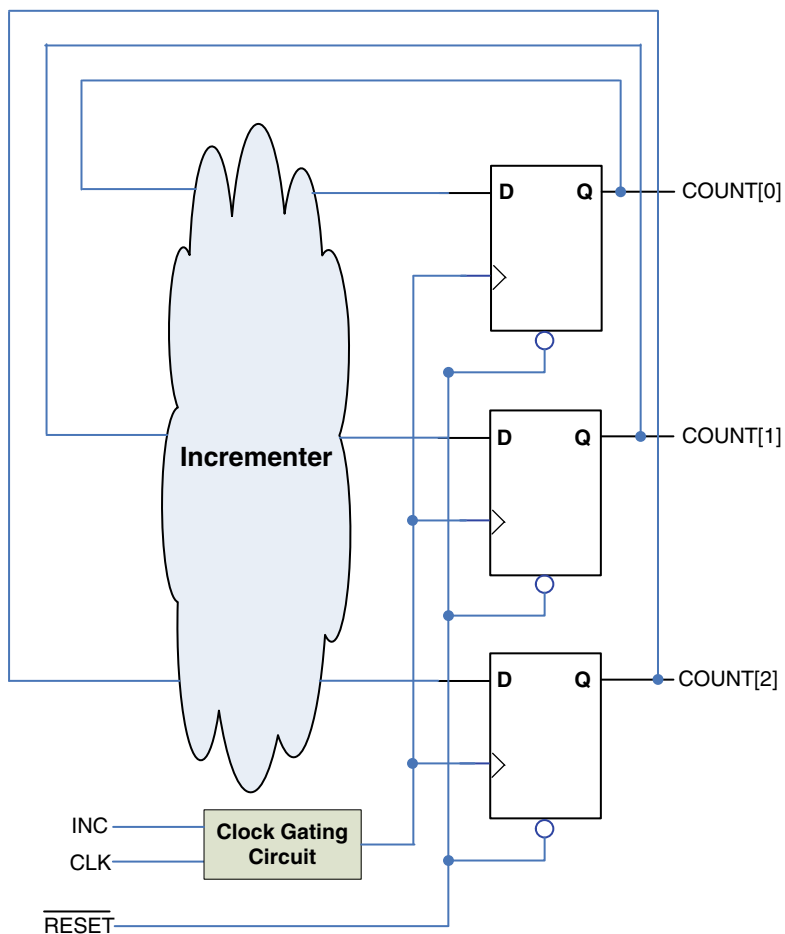


Fig. 2.22 Three bit counter with clock gating

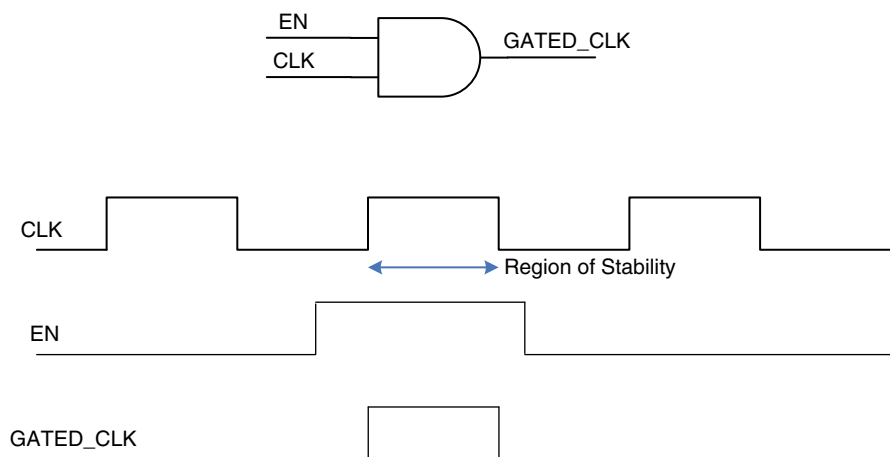


Fig. 2.23 Latch free clock gating circuit

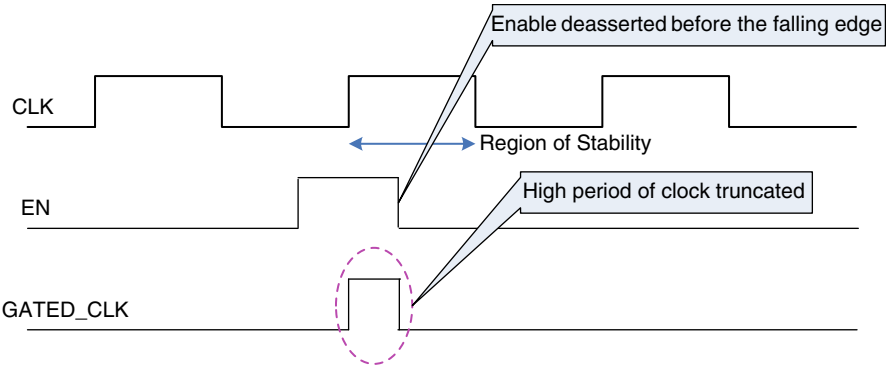


Fig. 2.24 Generated clock terminated prematurely

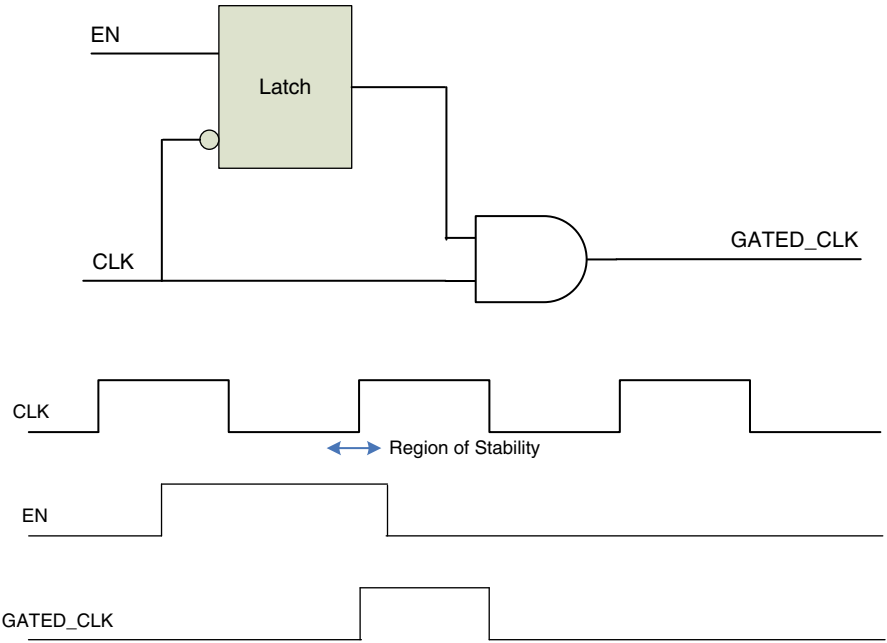


Fig. 2.25 Latch based clock gating circuit

2.5.2 Latch Based Clock Gating Circuit

The latch-based clock gating style adds a level-sensitive latch to the design to hold the enable signal from the active edge of the clock until the inactive edge of the clock, making it unnecessary for the circuit to itself enforce that requirement as shown in Fig. 2.25.

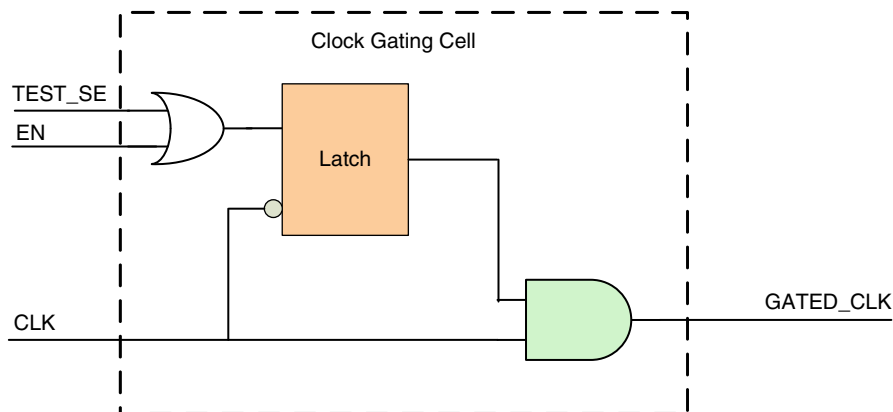


Fig. 2.26 Standard clock gating cell

Since the latch captures the state of the enable signal and holds it until the complete clock pulse has been generated, the enable signal need only be stable around the rising edge of the clock.

Using this technique, only one input of the gate that turns the clock on and off changes at a time, ensuring that the circuit is free from any glitches or spikes on the output.

Note: Use an AND gate to gate a clock that is active on the rising edge. For a clock that is active on the falling edge, use an OR gate to gate the clock and register the enable with a positive edge-triggered Latch.

When using this technique, special attention should be paid to the duty cycle of the clock and the delay through the logic that generates the enable signal, because the enable signal must be generated in half the clock cycle. This situation might cause problems if the logic that generates the enable command is particularly complex, or if the duty cycle of the clock is severely unbalanced. However, being careful with the duty cycle and logic delay may be acceptable compared with the problems created by other methods of gating clocks.

To ensure high manufacturing fault coverage, it is necessary to make sure the clock gating circuit is full controllable and observable to use within a scan methodology. A controllability signal which causes all flip-flops in the design to be clocked, regardless of the enable term value, can be added to allow the scan chain to shift information normally.

This signal can be ORed in with the enable signal before the latch and can be connected to either a test mode enable signal which is asserted throughout scan testing or to a scan enable signal which is asserted only during scan shifting.

The modified circuit is shown in Fig. 2.26. Most of the ASIC vendors do provide this “Clock Gating Cell” as a part of their standard library cell.

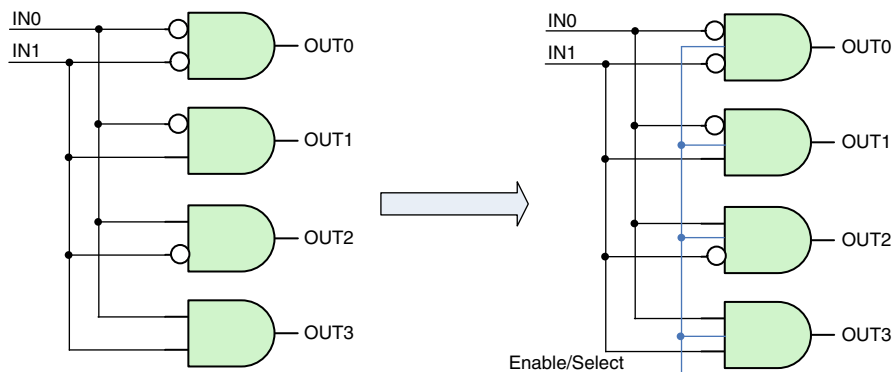


Fig. 2.27 Decoder with enable

2.5.3 Gating Signals

Effective power implementation can be achieved using gating signals for particular parts of the design. Similar to the concept of gating clock, signal gating reduces the transitions in clock free signals. The most common example is the decoder enable.

As part of an address decoding mechanism, signals used by other parts of the design may toggle as a reflection of activity in these parts. Switching activity on one input of the decoder will induce a large number of toggling gates. Controlling this with an enable or select signal prevents the propagation of their switching activity, even if the logic is slightly more complex (Fig. 2.27).

2.5.4 Data Path Re-ordering to Reduce Switching Propagation

Several data path elements, such as decoders or comparison operators, as well as “glitchy” logic may significantly contribute to power dissipation. The glitches, caused by late arrival signals or skews, propagate through other data path elements and logic until they reach a register. This propagation burns more power as the transitions traverse the logic levels. To reduce this wasted dissipation, designers need to rewrite the HDL code and shorten the propagation paths as much as possible. Figure 2.28 illustrates two implementations of the priority mux where the “glitchy” and “stable” conditions are ordered differently.

2.6 Reset Design Strategy

Many design issues must be considered before choosing a reset strategy for an ASIC design, such as whether to use synchronous or asynchronous resets, will every flip-flop receive a reset etc.

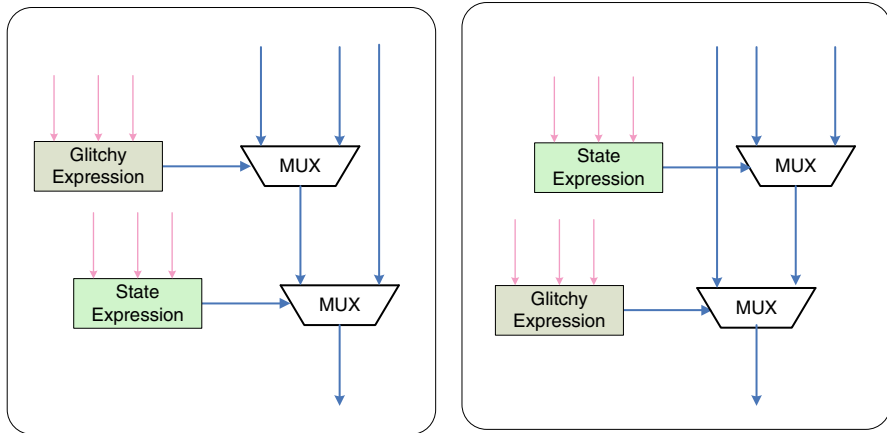


Fig. 2.28 Data path re-ordering to reduce switching propagation

The primary purpose of a reset is to force the SoC into a known state for stable operations. This would avoid the SoC to power on to a random state and get hanged. Once the SoC is built, the need for the SoC to have reset applied is determined by the system, the application of the SoC, and the design of the SoC. A good design guideline is to provide reset to every flip-flop in a SoC whether or not it is required by the system. In some cases, when pipelined flip-flops (shift register flip-flops) are used in high speed applications, reset might be eliminated from some flip-flops to achieve higher performance designs.

A design may choose to use either an Asynchronous or Synchronous reset or a mix of two. There are distinct advantages and disadvantages to use either synchronous or asynchronous resets and either method can be effectively used in actual designs. The designer must use an approach that is most appropriate for the design.

2.6.1 Design with Synchronous Reset

Synchronous resets are based on the premise that the reset signal will only affect or reset the state of the flip-flop on the active edge of a clock. In some simulators, based on the logic equations, the logic can block the reset from reaching the flip-flop. This is only a simulation issue, not a real hardware issue.

The reset could be a “late arriving signal” relative to the clock period, due to the high fanout of the reset tree. Even though the reset will be buffered from a reset buffer tree, it is wise to limit the amount of logic the reset must traverse once it reaches the local logic.

Figure 2.29 shows one of the RTL code for a loadable Flop with Synchronous Reset. Figure 2.30 shows the corresponding hardware implementation.

```

module load_syn_ff ( clk, in, out, load, rst_n);
input clk, in, load, rst_n;
output out;

always @(posedge clk)
    if (!rst_n)
        out <= 1'b0; // sync reset
    else if (load)
        out <= in; // sync

endmodule

```

Fig. 2.29 Verilog RTL code for loadable flop with synchronous reset

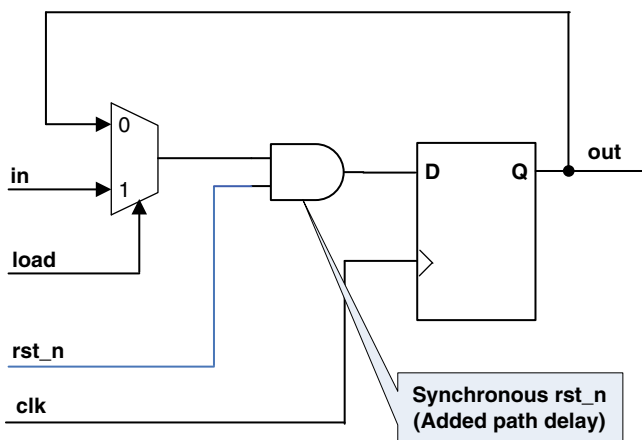


Fig. 2.30 Loadable flop with synchronous reset (hardware implementation)

One problem with synchronous resets is that the synthesis tool cannot easily distinguish the reset signal from any other data signal. The synthesis tool could alternatively have produced the circuit of Fig. 2.31.

Circuit shown in Fig. 2.31 is functionally identical to implementation shown in Fig. 2.30 with the only difference that reset AND gates are outside the MUX. Now, consider what happens at the start of a gate-level simulation. The inputs to both legs of the MUX can be forced to 0 by holding “*rst_n*” asserted low, however if “*load*” is unknown (X) and the MUX model is pessimistic, then the flops will stay unknown (X) rather than being reset. Note this is only a problem during simulation. The actual circuit would work correctly and reset the flop to 0.

Synthesis tools often provide compiler directives which tell the synthesis tool that a given signal is a synchronous reset (or set). The synthesis tool will “pull” this signal as close to the flop as possible to prevent this initialization problem from occurring.